Laboratorio 5 - Grupo 1: HTTP sin canal seguro

Guía general de reproducción - 8 de noviembre de 2026.

Descargar la guía completa en PDF. En GitHub, abre el archivo y utiliza **Download raw file / Descargar archivo** si el visor no lo muestra.

Este laboratorio demuestra que el inicio de sesión real de Vault tv expone sus datos cuando el conector **cliente de prueba** → **Core** utiliza HTTP. El Grupo 1 prepara ese contraejemplo y obtiene una captura de paquetes. Los grupos 2 y 3 utilizarán el mismo recorrido para configurar TLS y comprobar su protección.

El perfil HTTP es deliberadamente inseguro y exclusivo del laboratorio. No es una configuración de entrega del producto. No se modifican los defaults de Core, la autenticación, CSRF ni el hashing de contraseñas.

1 Alcance y aislamiento

La base de código es `develop` en el commit `1d8c562a55cab419d8c95657c399932d5ddc01ce`. La rama exclusiva es `feature/lab-05-secure-channel-g1`. Cada participante utilizará un clon nuevo en la carpeta que elija. No se debe ejecutar este perfil desde la carpeta del despliegue personal ni incorporar el perfil inseguro mediante un PR hacia `develop`.

Pieza	Función en el ensayo
Core	APIs reales de registro, sesión y perfil.
PostgreSQL	Cuenta ficticia y datos exclusivos del laboratorio.
Cliente	Ejecuta solicitudes reproducibles mediante `curl`.
Observador	Captura con `tcpdump`; `tshark` reconstruye y verifica las conversaciones.

El helper utiliza un nombre de proyecto Compose generado, una red y volúmenes propios. Core se publica exclusivamente en `127.0.0.1:18081`; PostgreSQL no publica un puerto del equipo. El despliegue normal de Vault tv no interviene: no se utiliza Caddy ni el API Gateway en este conector.

El observador comparte el espacio de red de Core y filtra TCP `8081`. Se eliminan las capacidades por defecto: conserva `NET_RAW` para capturar sin modo promiscuo y `DAC_OVERRIDE` para escribir la evidencia en el directorio privado montado desde Linux. No usa `NET_ADMIN`, modo privilegiado ni la red del host. La prueba supone acceso autorizado al trayecto del tráfico. Estar conectado a la misma red Wi-Fi no permite, por sí solo, leer las conversaciones unicast de otros equipos.

2 Herramientas y carpeta de trabajo

Se necesitan Git, acceso al repositorio, Docker Desktop con contenedores Linux y Docker Compose compatible con `up --wait`, Python 3.10 o posterior y Wireshark para la inspección visual. La primera construcción requiere conectividad para descargar imágenes y dependencias, y espacio libre en Docker. No se necesitan un IDE, Java instalado en Windows ni herramientas de captura instaladas dentro de Ubuntu: se construyen en las imágenes del laboratorio.

Abre Docker Desktop y espera a que su motor esté disponible. Si ejecutas desde Ubuntu, habilita su integración en Docker Desktop.

Abre PowerShell en cualquier carpeta donde quieras guardar un clon nuevo. Primero comprueba las herramientas:

```
git --version
python --version
docker compose version
docker info
```

`docker info` debe mostrar información del servidor sin error de conexión. Si `python` abre la tienda o no está disponible, configura Python o comprueba `py -3 --version` y utiliza `py -3` en todos los comandos posteriores.

Clonación en Windows / PowerShell

Clona la rama publicada. Si GitHub solicita autenticación, necesitarás acceso al repositorio; disponer del PDF no concede ese acceso. La variable evita partir el comando al copiarlo:

```
$labBranch = 'feature/lab-05-secure-channel-g1'
$labRepo = 'https://github.com/SWARCH-1A/streaming.git'
git clone --single-branch --branch $labBranch $labRepo streaming-lab5
cd streaming-lab5
git branch --show-current
git rev-parse HEAD
```

La rama debe ser `feature/lab-05-secure-channel-g1`. Registra el hash completo: identifica tu ejecución. Si la carpeta `streaming-lab5` ya existe, elige otro nombre para el clon; no borres el anterior. Comprueba que exista `labs/secure-channel/lab.py` y continúa con la sección 3. Cada instalación genera su cuenta ficticia y sus secretos. Para comparar HTTP y HTTPS conserva su propia `.state/` y sus volúmenes. No copies `.state/` entre compañeros.

Alternativa Ubuntu / WSL

Con Docker disponible desde la distribución, abre una terminal en la carpeta donde quieras clonar:

```
lab_branch=feature/lab-05-secure-channel-g1
lab_repo=https://github.com/SWARCH-1A/streaming.git
git clone --single-branch --branch "$lab_branch" "$lab_repo" streaming-lab5
cd streaming-lab5
git branch --show-current
git rev-parse HEAD
python3 --version
docker compose version
docker info
```

Sustituye `python` por `python3` en los pasos siguientes. Ejecuta todo desde la misma terminal y clon. La ejecución HTTP acreditada se realizó desde Windows con Docker Desktop; esta variante explica el uso compatible y no acredita una reproducción independiente en todas las plataformas.

3 Inicializar construir e iniciar

Desde la raíz del clon `streaming-lab5`, ejecuta cada comando por separado. Si uno falla, no continúes y consulta la sección 9:

```
python labs/secure-channel/lab.py init
python labs/secure-channel/lab.py build
python labs/secure-channel/lab.py up --mode http
python labs/secure-channel/lab.py status
```

Comando	Resultado esperado
`init`	Genera proyecto, secretos y cuenta ficticia en `.state/`. Repetirlo conserva el estado existente.
`build`	Construye Core con su Dockerfile existente y la imagen de herramientas del ensayo.
`up --mode http`	Inicia la base y Core; espera la respuesta de salud `UP`.
`status`	Muestra los contenedores propios del laboratorio.

La dirección local es `http://localhost:18081`. Core ofrece APIs: no se despliega aquí la Web de Vault tv. El cliente del ensayo habla con `http://core-lab:8081` dentro de la red Docker. Por eso el puerto visible en el PCAP es **8081**, aunque Windows accede mediante **18081**.

En el perfil HTTP no se genera ni se instala un certificado de confianza. La preparación del certificado corresponde al Grupo 2 para el perfil HTTPS.

Si el arranque falla, no continúes suponiendo que Core está listo:

```
python labs/secure-channel/lab.py logs
```

4 Crear la cuenta ficticia

```
python labs/secure-channel/lab.py seed
```

El helper llama a `POST /api/identity/registrations` con correo del dominio reservado `example.test`, handle y contraseña ficticios. La contraseña contiene el marcador `LAB5_CANAL_INSEGURO`, seguido de una parte aleatoria. Se conserva el UUID de idempotencia y el mismo cuerpo para que repetir `seed` no cree otra cuenta.

El resultado esperado es registro `201`. La cuenta y sus IDs quedan guardados exclusivamente en `.state/`. El registro se realiza **antes de iniciar la captura principal** para que la evidencia se concentre en autenticación y consulta del perfil.

No publiques `.state/credentials.json`, cookies, cabeceras, PCAP, claves o archivos de configuración privada. Aunque todos los datos sean ficticios, una sesión del ensayo sigue siendo una credencial durante su vigencia. `.state/` está ignorado por Git.

5 Capturar el recorrido real

```
python labs/secure-channel/lab.py capture --mode http
python labs/secure-channel/lab.py verify --mode http
```

`capture` espera a que `tcpdump` esté escuchando, ejecuta el cliente y detiene la captura de forma controlada. El cliente realiza:

1. `GET /api/identity/csrf`: obtiene token y cookie CSRF.
2. `POST /api/identity/sessions`: envía JSON con `login` **y** `password`, la cookie y la cabecera `X-XSRF-TOKEN`.
3. Conserva la cookie de sesión `stream_session` obtenida en el login.
4. `GET /api/profile/me`: utiliza esa sesión para consultar el perfil.

Se esperan respuestas `200` y el mismo `userId` que produjo el registro. El script no imprime la contraseña ni la cookie en la terminal.

`verify` comprueba la captura TCP reconstruida, las credenciales ficticias exactas, el éxito del login, la sesión utilizada por la consulta de perfil y la identidad de sus respuestas. Un marcador aislado o un `200` de salud no bastan para aprobar el ensayo.

Los campos `reassembledLogin`, `exactFictitiousCredentialsReadable`, `csrfPreserved`, `sessionCookieReadable` y `authenticatedProfileMatches` deben aparecer en `true`. Si el comando termina con error o falta alguna comprobación, la ejecución no está aprobada.

Archivos locales producidos

Todos se guardan bajo `labs/secure-channel/.state/`:

Archivo	Uso
`http/traffic.pcap`	Evidencia de red que se abre en Wireshark.
`http/capture.json`	Fecha UTC, referencia de código y hash de la captura.
`http/verification.json`	Comprobaciones realizadas sobre el PCAP.
`http/follow-streams.txt`	Conversaciones reconstruidas para inspección local. Contiene credenciales ficticias.
`http/client-result.json`	Resultado funcional del cliente.
`http/tcpdump.log`	Diagnóstico del observador.

Repetir `capture` en el mismo modo reemplaza la captura y sus resultados. Si debes conservar una ejecución, copia primero su carpeta `http` a otra ubicación privada fuera del repositorio. No uses los archivos de una ejecución anterior para describir como aprobada una ejecución nueva que haya fallado.

6 Inspeccionar la captura en Wireshark

1. Abre Wireshark en Windows.

2. Pulsa **Ctrl + O** o selecciona **Archivo** → **Abrir / File** → **Open**. Navega a tu clon y abre `labs/secure-channel/.state/http/traffic.pcap`. Es una ruta relativa al clon, no una ubicación fija del computador. Si clonaste dentro de WSL, utiliza el acceso Linux del Explorador para llegar a la distribución y carpeta; muestra archivos ocultos si no ves `.state`.

3. En el filtro de visualización introduce:

```
tcp.port == 8081
```

4. Si no se reconocen las peticiones HTTP, selecciona un paquete del flujo y utiliza **Analyze** → **Decode As** → **HTTP** para el puerto TCP `8081`.

5. Localiza la petición `POST /api/identity/sessions`. Después de decodificar HTTP, utiliza este filtro:

```
http.request.method == "POST"
```

6. Haz clic derecho y selecciona **Seguir** → **Flujo TCP / Follow** → **TCP Stream**. Selecciona conversación completa y formato ASCII o UTF-8. El diálogo reconstruye la conversación aunque el JSON esté repartido entre paquetes.

7. Identifica método, ruta, JSON con `login` y `password`, marcador `LAB5_CANAL_INSEGURO`, respuesta `200` y cabecera `Set-Cookie` de `stream_session`.

8. Cierra la ventana de seguimiento, limpia el filtro aplicado por Follow y localiza `GET /api/profile/me`. Comprueba la cookie enviada y el perfil devuelto. El perfil puede pertenecer a otro flujo TCP porque cada solicitud de `curl` abre su conexión.

```
http.request.uri == "/api/profile/me"
```

No necesitas mantener los servidores encendidos para abrir el PCAP guardado. No pulses «Iniciar captura» ni selecciones Wi-Fi: el observador ya generó el archivo. Usa 8081, no 18081, como filtro. La captura de pantalla de `curl` no sustituye esta evidencia: muestra qué quiso enviar el cliente. El PCAP y Follow TCP Stream permiten observar qué información pudo reconstruir el observador del trayecto.

Para compartir capturas en el informe, oculta la contraseña completa, los tokens y las cookies. Conserva visible el marcador, las rutas y los estados HTTP necesarios para explicar el resultado. No distribuyas el PCAP públicamente.

7 Qué demostrar y cómo explicarlo

Comprobación	Qué permite concluir
Registro `201`, login `200`, perfil `200` e identidad coincidente	Se utilizó un recorrido funcional del sistema, no un servidor de eco.
JSON de login legible en el flujo TCP	HTTP expone las credenciales de la cuenta ficticia en tránsito.
Cookie de sesión legible y reutilizada en el perfil	También queda expuesta la credencial de sesión transportada por HTTP.
Token y cookie CSRF conservados	La exposición no depende de haber desactivado CSRF.
Captura restringida al conector del laboratorio	La demostración tiene un alcance concreto y autorizado.

RNF-025 exige HTTPS/TLS para credenciales, tokens y datos privados en comunicaciones accesibles por red. Este perfil constituye el contraejemplo deliberado que el laboratorio busca corregir. RNF-026 exige hashing apropiado de contraseñas en almacenamiento: Core mantiene ese mecanismo, pero el hash de la base no cifra el trayecto cliente-servidor.

CSRF protege frente a determinadas solicitudes no autorizadas desde otros sitios; HttpOnly restringe el acceso de scripts a la cookie. Ninguno cifra los paquetes ni sustituye TLS.

Explicación para la demostración:

El cliente inicia sesión en Core usando su API real. La cuenta y la sesión funcionan, pero el transporte HTTP permite reconstruir el identificador, la contraseña ficticia y la cookie. El hashing protege la contraseña almacenada; TLS es necesario para proteger su tránsito. Esta observación requiere acceso al trayecto de red, como el observador autorizado que preparamos en Docker.

El Grupo 1 no demuestra ataques activos, acceso a redes ajenas ni seguridad de todo Vault tv. La comparación TLS posterior debe evaluar confidencialidad, integridad y autenticación del servidor para este mismo conector; no constituye una prueba de disponibilidad.

8 Entrega a los grupos 2 y 3

Antes de comparar resultados, los grupos deben confirmar el mismo conector, estructura del cuerpo de login, endpoints y punto de captura. Cada instalación conserva su propia cuenta para comparar HTTP con HTTPS; no se comparte una cuenta entre computadores. El perfil HTTPS conservará el puerto interno `8081` y el publicado `18081`; los modos se ejecutan de manera excluyente.

Grupo	Responsabilidad
Grupo 1	Perfil HTTP aislado, cuenta ficticia, captura, análisis y reproducción de la exposición.
Grupo 2	Generar llaves y certificado con OpenSSL; configurar TLS directo en Core y la confianza del cliente.
Grupo 3	Repetir el recorrido por HTTPS; comprobar tráfico TLS y rechazo de certificados no confiables o nombres incorrectos.

Core ya permite configurar TLS público mediante `CORE_PUBLIC_TLS_ENABLED`, `CORE_PUBLIC_TLS_KEYSTORE`, `CORE_PUBLIC_TLS_KEYSTORE_PASSWORD` y `CORE_SECURE_COOKIE`. Su almacén es **PKCS#12**: no basta con montar únicamente un certificado PEM y una llave.

El helper reserva esta interfaz local para el material del Grupo 2:

```
labs/secure-channel/.state/tls/
core.p12
ca.crt
keystore-password.txt
```

El certificado debe incluir los SAN DNS `core-lab` y `localhost`, y el SAN IP `127.0.0.1`. El usuario `10001` de Core debe poder recorrer el directorio montado y leer `core.p12`; el montaje es de solo lectura. En Linux/WSL, `init` crea el directorio privado `tls` con permisos restrictivos: el Grupo 2 debe preparar permisos de recorrido del directorio y lectura del almacén para ese UID. No basta con ajustar únicamente el archivo. La contraseña del almacén se guarda en `keystore-password.txt`, sin espacios añadidos; la lee el helper anfitrión, no Core directamente. Conserva restringidos ese archivo y cualquier clave privada PEM. La clave privada y la contraseña no se comparten por Git.

Cuando el Grupo 2 haya preparado y revisado esos archivos:

```
python labs/secure-channel/lab.py up --mode https
python labs/secure-channel/lab.py capture --mode https
python labs/secure-channel/lab.py verify --mode https
```

El cliente usa explícitamente `--cacert /state/tls/ca.crt`; no desactiva la validación mediante `-k`. No es necesario importar la CA en Windows para ejecutar este cliente dentro de Docker. Una prueba adicional desde un navegador requerirá el procedimiento de confianza correspondiente.

Estos comandos describen el soporte preparado para la comparación. No afirman que la generación del certificado ni la aceptación TLS de los otros grupos ya se hayan ejecutado. El Grupo 3 debe comprobar operación funcional, handshake y registros TLS, además de los casos negativos de CA y nombre. La mera ausencia del marcador no prueba que el canal sea seguro. Wireshark no debe tener cargadas claves de sesión TLS durante esa comparación.

9 Diagnóstico y cierre

Síntoma	Comprobación
Docker no responde	Abrir Docker Desktop; revisar integración WSL si corresponde.
Python no existe o abre la tienda	Configurar Python; comprobar `py -3 --version` como alternativa.
Git deniega acceso	Comprobar permisos del repositorio y autenticación. No compartir tokens.
Compose no reconoce `--wait`	Actualizar Docker Desktop/Compose a una versión compatible.
`Run init first`	Ejecutar `init` desde la raíz del clon dedicado.
Puerto `18081` ocupado	Identificar el proceso que lo utiliza. No detener otros despliegues sin comprobar su propietario.
Build falla	Revisar el primer error y la conectividad para descargar imágenes y dependencias.
Core no queda listo	Ejecutar `logs`; revisar migraciones, conexión a la base y variables. No omitir readiness.
`capture` pide `seed`	Preparar primero la cuenta ficticia mediante `seed`.
Falta tráfico o marcador	Verificar que se abrió el PCAP nuevo y que la captura terminó. Usar `8081`, no `18081`, en el filtro de esta captura.
La verificación falla	No presentar la ejecución como aprobada. Conservar el error y revisar el flujo que falta.
HTTPS exige archivos	Coordinar la generación PKCS#12 y CA con el Grupo 2. No usar `-k` para ocultar el problema.

Para detener únicamente los contenedores del laboratorio y conservar datos y evidencia:

```
python labs/secure-channel/lab.py down
```

Para repetir otro día sin reconstruir:

```
python labs/secure-channel/lab.py up --mode http
python labs/secure-channel/lab.py seed
python labs/secure-channel/lab.py capture --mode http
python labs/secure-channel/lab.py verify --mode http
python labs/secure-channel/lab.py down
```

No uses `down -v`, borres volúmenes ni elimines `.state/` durante el recorrido. Si cambias el código de Core o la imagen de herramientas, ejecuta `build` antes de repetirlo.

Evidencia para la entrega

Cada participante completa su ejecución. La revisión por otra persona es una comprobación posterior, no un requisito para ejecutar el trabajo completo del Grupo 1.

Dato o comprobación	Resultado observado
Fecha, sistema operativo y terminal	_____
Commit: `git rev-parse HEAD`	_____
Versiones de Python, Docker y Compose	_____
Versión de Wireshark	_____
Build y Core UP	Aprobado / Fallido / No probado
Registro 201; CSRF, login y perfil 200	Aprobado / Fallido / No probado
Todos los checks de verify HTTP en true	Aprobado / Fallido / No probado
Login legible en Follow TCP Stream	Aprobado / Fallido / No probado
Cookie y perfil de la misma sesión	Aprobado / Fallido / No probado
Hash del PCAP: campo pcapSha256 de verification.json	_____
Parada conservando datos	Aprobado / Fallido / No probado
Error, conclusión y evidencia redactada	_____

El hash de otra captura será diferente aunque el recorrido pase: cada instalación y sesión produce su tráfico. No distribuyas el PCAP, las transcripciones ni `.state/` con el informe público.

Antecedentes de validación: el recorrido HTTP, la conservación de datos tras reiniciar y 12 pruebas del laboratorio se aprobaron en Windows con Docker Desktop. La captura acreditada corresponde a `606fbbbbecbbd11316ccf0dde902cbd41f1849fd`; `da1cbaad8bb64e9c2d7aafcf9491949477a33470` añadió documentación. Esta edición general cambia solo documentación. No atribuye aprobación a la ejecución de otro participante, a la inspección visual pendiente ni a la comparación TLS. Registra siempre el commit de tu clon.

Referencias

- SPEC-01 Cuentas y sesión.
- Catálogo de requisitos RNF-025 y RNF-026.
- Configuración actual de Core.
- Follow TCP Stream en Wireshark.
- OpenSSL PKCS12.
- TLS 1.3 RFC 8446.